

Task 1 – Kubernetes Security Hardening

1. Các Lỗi hỏng, Cấu hình sai và Rủi ro cũng như Mức độ ảnh hưởng

Dựa trên workload mẫu là một ứng dụng Backend API (Python/Flask), quá trình đánh giá đã phát hiện các cấu hình sai sau:

ID	Phát hiện (Finding)	Mức độ	Rủi ro (Risk)	Tác động khi bị xâm nhập (Impact)
F-01	Hardcoded Secrets Chứa <code>`DB_PASSWORD`</code> trực tiếp trong env var của Deployment.	High	Thông tin đăng nhập cơ sở dữ liệu bị lộ cho bất kỳ ai có quyền đọc manifest hoặc truy cập API Server.	Kẻ tấn công chiếm quyền truy cập CSDL, đánh cắp/xóa dữ liệu nhạy cảm.
F-02	HostPath Mount `/ Mount thư mục gốc của Node host vào container.	Critical	Container có khả năng đọc/ghi toàn bộ hệ thống tệp tin của máy chủ vật lý/máy ảo.	Container Escape: Ghi SSH key vào <code>`/root/.ssh`</code> , chiếm quyền kiểm soát Node và toàn bộ Cluster.
F-03	No Resource Limits Không khai báo <code>`requests`</code> và <code>`limits`</code> cho CPU/RAM.	Medium	Một container lỗi hoặc bị tấn công DDoS có thể chiếm dụng hết tài nguyên của node.	Denial of Service (DoS): Các Pod khác trên cùng node bị treo, ảnh hưởng tính sẵn sàng.
F-04	Run as Root / Privilege Escalation Sử dụng default security context (chạy với UID 0).	Medium	Tăng diện tích tấn công nếu xuất hiện lỗi hỏng khai thác Kernel.	Dễ dàng chuyển đổi từ User space sang Kernel space; leo quyền và vượt qua cách ly cơ bản.
F-05	Mutable Image Tag (`latest`) Sử dụng tag không bất biến.	Low	Không xác định chính xác phiên bản mã nguồn đang chạy; rủi ro Supply Chain Attack.	Hacker đẩy mã độc lên tag <code>`latest`</code> . Khi Pod khởi động lại, mã độc sẽ tự động tải về.

2. Cách thức phát hiện (Detection Methodology)

Các lỗi hỏng trên được phát hiện bằng cách kết hợp:

- Static Scanning (Quét tĩnh):** Sử dụng các công cụ rà quét IaC (Infrastructure as Code) như `Trivy` quét trực tiếp trên file YAML của ứng dụng.
- Manual Review:** Đối chiếu file manifest với các tiêu chuẩn bảo mật của CIS Kubernetes Benchmark.

3. Khắc phục (Remediation)

Phiên bản manifest đã được hardening (``deployment-secure.yaml``) với các thay đổi chính:

3.1. Quản lý Bí mật (Secrets Management)

- **Thay đổi:** Loại bỏ ``value:`` chứa mật khẩu thô; thay bằng ``secretKeyRef`` tham chiếu đến đối tượng ``Secret`` độc lập.

3.2. Quyền tối thiểu & Thắt chặt Container (Least Privilege & Hardening)

- **Security Context:**
 - ``runAsUser: 10001``, ``runAsGroup: 10001``: Ép buộc chạy dưới quyền tài khoản non-root.
 - ``allowPrivilegeEscalation: false``: Ngăn chặn tiến trình con gia tăng quyền hạn.
 - ``readOnlyRootFilesystem: true``: Chặn mã độc ghi tập tin vào filesystem của container.
 - ``capabilities.drop: ["ALL"]``: Gỡ bỏ toàn bộ Linux Capabilities không cần thiết.
- **Volume Security:**
 - Xóa hoàn toàn cấu hình ``hostPath``.

Thay thế bằng ``emptyDir`` mount vào ``/tmp`` để phục vụ nhu cầu ghi tạm (do rootfs ở chế độ read-only).

3.3. Quản lý Tài nguyên & Hình ảnh (Resources & Stability)

- **Resource Quotas:** Khai báo rõ ràng ``requests`` và ``limits`` cho CPU/Memory nhằm ngăn chặn hiện tượng Noisy Neighbor và lỗi OOM Kill.
- **Immutable Images:** Cố định phiên bản image cụ thể (ví dụ: ``python:3.9-slim``) thay vì sử dụng tag ``latest``.

3.4. Xác minh (Verification)

Sau khi deploy bản sửa lỗi, các bước kiểm chứng sau đã được thực hiện thành công:

1. ``kubectl exec -it <pod-name> -- whoami`` → Trả về ``10001`` (Xác nhận quyền Non-root).
2. Thực hiện ``touch /test.txt`` trong container → Báo lỗi ``Read-only file system`` (Xác nhận cơ chế bảo vệ Filesystem).

Ứng dụng phản hồi HTTP ``200 OK`` (Xác nhận tính toàn vẹn chức năng).

4. Phòng tránh tái diễn (Prevention Strategy)

Để đảm bảo các lỗi tương tự không xuất hiện ở các dự án khác, đề xuất đưa bảo mật vào quy trình "Shift-Left":

4.1. Tích hợp CI/CD Pipeline (Gatekeeping)

Tích hợp scanner vào GitHub Actions/GitLab CI trước bước Deploy:

None

Ví dụ cấu hình GitHub Action

- name: Run Trivy Vulnerability Scanner

```
uses: aquasecurity/trivy-action@master
with:
  scan-type: 'config'
  scan-ref: '.'
  exit-code: '1' # Dừng pipeline nếu phát hiện lỗi
High/Critical
```

Quy tắc: Pipeline sẽ bị từ chối tự động nếu người phát triển commit các manifest chứa secret hoặc thiếu cấu hình giới hạn tài nguyên.

4.2. Admission Control Policy (Runtime Enforcement)

Sử dụng **OPA Gatekeeper** hoặc **Kyverno** để thiết lập các quy tắc Constraint bắt buộc trên toàn bộ Cluster:

- **Rule 1:** Cấm sử dụng image tag `'latest'`.
- **Rule 2:** Bắt buộc tất cả Pod phải cấu hình `'securityContext.runAsNonRoot: true'`.
- **Rule 3:** Nghiêm cấm mount volume loại `'hostPath'`.

Cơ chế hoạt động: Kubernetes API Server sẽ từ chối các yêu cầu `'kubectl apply'` vi phạm policy và phản hồi thông báo lỗi chi tiết cho kỹ sư phát triển.

5. Đề xuất Rollout trong Production (Operational Approach)

Để tránh rủi ro gián đoạn dịch vụ khi áp dụng policy trên hệ thống Production hiện hữu, lộ trình triển khai từng bước được đề xuất như sau:

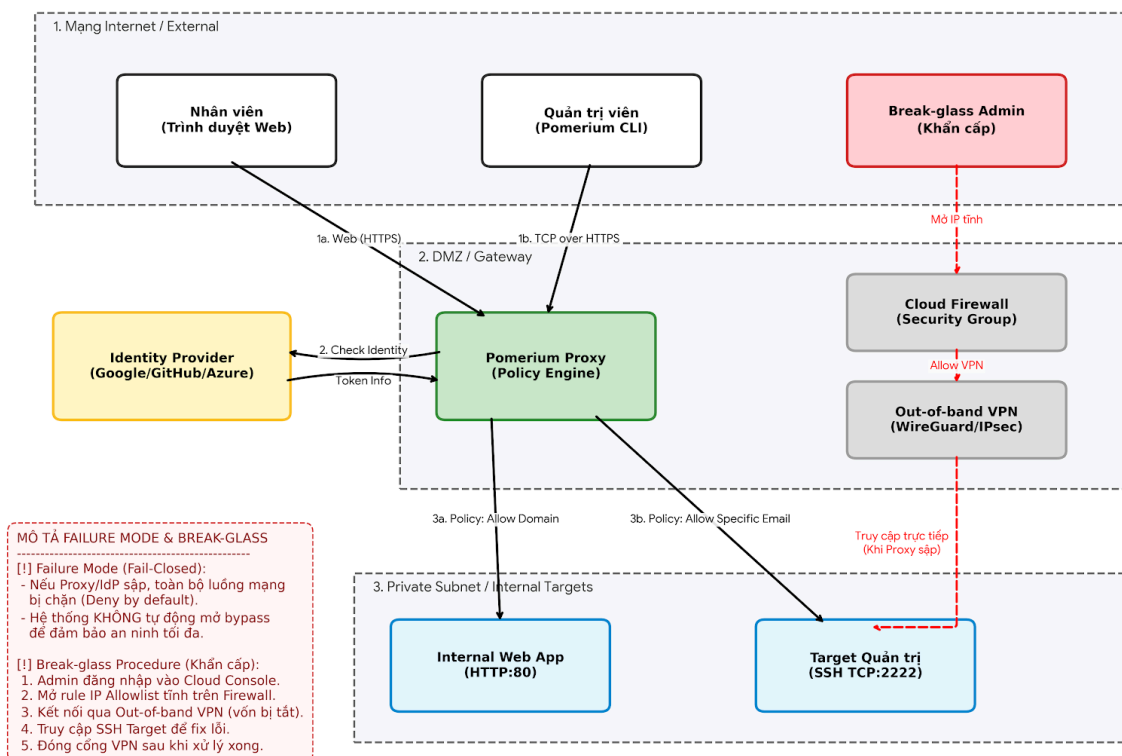
Giai đoạn	Hành động	Mục tiêu
1. Audit (Dry-run)	Cài đặt Kyverno/Gatekeeper ở chế độ Audit Only .	Ghi nhận thông tin vi phạm mà không chặn triển khai; đánh giá phạm vi ảnh hưởng (blast radius).
2. Prioritize	Phân loại các phát hiện theo mức độ nghiêm trọng (Critical > High > Medium).	Ưu tiên khắc phục các lỗi hổng nguy hiểm nhất (HostPath, Privileged) trước.
3. Gradual Enforce	Bật chế độ Enforce cho Namespace mới hoặc môi trường phụ. Cấp ngoại lệ (Exception) tạm thời cho hệ thống cũ.	Giảm thiểu rủi ro sự cố; tạo thời gian cho đội ngũ phát triển xử lý nợ kỹ thuật.

Giai đoạn	Hành động	Mục tiêu
4. Full Enforcement	Kích hoạt Enforce Mode trên toàn bộ Cluster và duy trì quy trình phê duyệt ngoại lệ chuẩn.	Đảm bảo tất cả các workload triển khai mới tuân thủ tiêu chuẩn bảo mật ngay từ đầu.

Task 2 – Secure Access Architecture: ZTNA

1. Sơ Đồ & Mô Tả Thiết Kế Tổng Quan

Sơ đồ Kiến trúc ZTNA & Identity-Aware Proxy (Kèm Mô tả Break-glass)



Mô hình Zero Trust Network Access (ZTNA) được thiết kế dựa trên Identity-Aware Proxy để bảo vệ hai mục tiêu chính: một **Internal Web App (HTTP)** và một **SSH Target (TCP)**. Kiến trúc không dựa vào mạng lưới (VLAN/VPN truyền thống) mà dựa vào danh tính (Identity) và ngữ cảnh để cấp quyền truy cập.

- **Gateway / Proxy:** Lựa chọn **Pomerium**. Người dùng từ Internet sẽ kết nối đến Pomerium.
- **Identity Provider (IdP):** Tích hợp với Google Workspace, Azure AD hoặc GitHub. Pomerium sẽ chuyển hướng luồng chưa xác thực sang IdP.
- **Policy Enforcement Point:** Sau khi IdP trả về danh tính (Identity) và các thuộc tính (Group, Domain), Pomerium sẽ đánh giá dựa trên tập luật (Policy-as-Code) để cho phép hoặc từ chối kết nối.
- **Targets:** Web App tiếp nhận luồng HTTP đã được ủy quyền. SSH Target tiếp nhận luồng TCP tunnel được bọc qua HTTPS từ Pomerium CLI.

2. Phân Tích Kỹ Thuật

2.1. Vì sao chọn Pomerium (so với oauth2-proxy)?

Pomerium được chọn vì tính năng hỗ trợ đa giao thức (Multi-protocol). Mặc dù oauth2-proxy hoạt động rất tốt với các ứng dụng Web (HTTP/HTTPS), việc bảo vệ cổng SSH với

oauth2-proxy yêu cầu thiết lập các luồng tunneling phức tạp hoặc các công cụ phụ trợ. Pomerium được xây dựng trên Envoy Proxy, hỗ trợ proxy TCP trực tiếp. Quản trị viên chỉ cần sử dụng lệnh `pomerium-cli tcp` để kết nối SSH thông qua luồng xác thực OIDC một cách tự nhiên và mượt mà.

2.2. Mô hình Phân quyền / Policy

Chính sách được thiết kế theo mô hình **RBAC kết hợp ABAC** và nguyên tắc **Least Privilege**:

- **Internal Web App**: Phân quyền rộng. Áp dụng policy cho phép dựa trên Domain (ví dụ: mọi tài khoản `*@company.com`) hoặc Group (ví dụ: nhóm Developers, Marketing).
- **SSH Target (Quản trị)**: Phân quyền cực kỳ nghiêm ngặt. Chỉ cho phép các email đích danh cụ thể (ví dụ: `admin-1@company.com`) của đội ngũ SysAdmin/DevOps.

Lý do: Việc tách biệt này giới hạn "blast radius" (phạm vi ảnh hưởng). Nếu thông tin đăng nhập của nhân viên thông thường bị lộ, kẻ tấn công cũng không thể truy cập vào các cổng quản trị hạ tầng (SSH).

2.3. Failure Mode & Break-Glass Procedure

Điều gì xảy ra khi Proxy hoặc IdP down? Quản trị viên truy cập khẩn cấp bằng cách nào?

- **Failure Mode (Fail-Closed)**: Nếu Pomerium sập, hoặc IdP (Google/GitHub) gặp sự cố, hệ thống sẽ từ chối mọi kết nối (Deny by default). Hệ thống không bao giờ tự động mở bypass để đảm bảo an ninh tối đa.
- **Break-glass (Quy trình khẩn cấp)**: Để quản trị viên có thể vào xử lý sự cố, hệ thống thiết kế một Out-of-band Bastion Host hoặc một cổng Site-to-Site/Client-to-Site VPN ẩn (như WireGuard) nằm độc lập hoàn toàn với Pomerium. Cổng này bình thường bị chặn/khóa (disabled) ở mức Cloud Firewall (AWS Security Group / Physical Firewall).
- **Thực thi Break-glass**: Quản trị viên nắm quyền Root của Cloud/Firewall sẽ đăng nhập vào Cloud Console, mở rule IP Allowlist tĩnh từ IP tĩnh của nhà quản trị để kết nối vào VPN. Việc này ngay lập tức kích hoạt cảnh báo P0 (Critical Alert) tới toàn đội ngũ. Sau khi xử lý xong, cổng VPN phải được đóng lại.

2.4. Điểm Yếu & Hướng Khắc Phục Ở Quy Mô Lớn

- **SPOF (Single Point of Failure)**: Triển khai Pomerium ở chế độ đơn lẻ dễ dẫn đến sập toàn hệ thống.
Khắc phục: Triển khai chế độ Distributed Mode (tách biệt Dataplane và Controlplane) trên Kubernetes hoặc AWS ECS, chạy nhiều replicas đằng sau Network Load Balancer đa vùng (Multi-AZ).
- **Session Hijacking**: Kẻ gian cướp cookie phiên.
Khắc phục: Cấu hình TTL của JWT ngắn (15-30 phút), buộc xác thực lại hoặc áp dụng Continuous Access Evaluation (CAE).
- **Malware trên thiết bị quản trị**: Tài khoản hợp lệ nhưng máy tính nhiễm mã độc.
Khắc phục: Tích hợp Device Identity (Thiết bị phải có chứng chỉ mTLS hợp lệ do công ty cấp) và Device Posture (Kiểm tra agent EDR như SentinelOne/CrowdStrike có báo cáo an toàn hay không trước khi cấp quyền).

3. Explicit Explanation

POC này áp dụng các kỹ thuật giả lập (mocking) như sau:

- **Mock DNS & TLS Certificate:**
 - *Cách làm:* Sử dụng file `/etc/hosts` để giả lập các domain nội bộ (`*.localhost.pomerium.io`) và bật cờ `autocert: true` trong Pomerium để tự động sinh Self-signed Certificate.
 - *Lý do:* Giúp bỏ qua bước đăng ký public domain và Let's Encrypt phức tạp, nhưng vẫn đảm bảo luồng giao tiếp được mã hóa (HTTPS/TLS) theo chuẩn Zero Trust.
- **Mock Internal Services (Target):**
 - *Cách làm:* Thay vì dùng các server nội bộ thật, POC sử dụng 2 lightweight Docker image: `kennethreitz/httpbin` (giả lập một web app nội bộ nhận HTTP request) và `lscr.io/linuxserver/openssh-server` (giả lập một máy chủ cần quản trị qua SSH TCP port 2222).
 - *Lý do:* Tiết kiệm tài nguyên, dễ dàng spin-up bằng docker-compose và cô lập hoàn toàn môi trường test.
- **Mock Identity Environment:**
 - *Cách làm:* Sử dụng Identity Provider thật (GitHub hoặc Google OAuth) nhưng Callback URL được trỏ về `localhost`.
 - *Lý do:* Việc này mô phỏng chính xác luồng SSO (Single Sign-On) của doanh nghiệp nhưng không yêu cầu phải thiết lập một Enterprise IdP (như Azure AD hay Okta) trả phí.

File 1: `docker-compose.yml`

```
None
version: '3.7'

services:
  # Pomerium - Zero Trust Identity-Aware Proxy
  pomerium:
    image: pomerium/pomerium:latest
    ports:
      - "443:443"
      - "80:80"
    volumes:
      - ./config.yaml:/pomerium/config.yaml:ro
    environment:
      # Mock secrets for POC demonstration
```

```

-
SHARED_SECRET=Z2VuZXJhdGVfc29tZV9yYW5kb21fc2VjcmV0X2hlcmU=
-
COOKIE_SECRET=Z2VuZXJhdGVfc29tZV9yYW5kb21fY29va2l1X2hlcmU=

# Target 1: Internal Web Application (HTTP)
webapp:
  image: kennethreitz/httpbin
  expose:
    - "80"

# Target 2: SSH Server for System Administration (TCP)
ssh-server:
  image: lscr.io/linuxserver/openssh-server:latest
  environment:
    - USER_NAME=admin
    - USER_PASSWORD=secretpassword
  expose:
    - "2222"

```

File 2: config.yaml

```

None
# Identity Provider Configuration (Example using GitHub)
authenticate_service_url: https://auth.localhost.pomerium.io
idp_provider: "github"
idp_client_id: "YOUR_GITHUB_OAUTH_CLIENT_ID"
idp_client_secret: "YOUR_GITHUB_OAUTH_CLIENT_SECRET"

# Use local self-signed certificates for POC
autocert: true

```

```
routes:
```

```
  # Route 1: Internal Web App (Broad Access)
```

```
  - from: https://webapp.localhost.pomerium.io
```

```
    to: http://webapp:80
```

```
    policy:
```

```
      - allow:
```

```
        or:
```

```
          - email:
```

```
            is: user@yourdomain.com # Thay bằng email dùng để
```

```
test
```

```
  # Route 2: SSH Target (Strict Least Privilege Access)
```

```
  - from: tcp+https://ssh.localhost.pomerium.io
```

```
    to: tcp://ssh-server:2222
```

```
    policy:
```

```
      - allow:
```

```
        and:
```

```
          - email:
```

```
            is: root-admin@yourdomain.com # Thay bằng email
```

```
admin
```

Task 3 – Malware Reversing

1. Tổng quan

Thông tin Mẫu (Sample Metadata)

- File: `Malware.exe`
- SHA-256: `e91a4af62987547438341666daa943a1ce1a5dd0be0a5ce82111849912bf4e46`
- PE: 32-bit Windows PE executable
- Architecture: x86
- Subsystem: Windows Console
- Compiler/Linker: Microsoft Visual C++ 14.x

Bắt bắt lưu lượng mạng (Network Capture)

- File: `pcap.pcapng`
- Sample host: `192.168.50.250`
- Destination: `192.168.50.175:9999/TCP`

Phương pháp Phân tích

Phân tích được thực hiện bằng cách kết hợp:

- **Static analysis:** PE inspection, strings, import table và reverse engineering các routine liên quan đến file system, cryptography và network.
- **Network analysis:** phân tích TCP stream trong Wireshark/PCAP để xác định destination, protocol và dữ liệu được truyền.

2. Phân tích Hành vi Mẫu

2.1. Tìm thư mục Desktop

Sample gọi:

```
None  
SHGetSpecialFolderPathW (CSIDL = 0)
```

Sau đó sample sử dụng:

```
None  
FindFirstFileW
```

FindNextFileW

để duyệt các file trong thư mục.

Các Định dạng Tập Mục tiêu

Sample chứa danh sách:

- Hình ảnh: **.png, .jpg, .bmp**
- Văn bản & Tài liệu: **.doc, .docx, .pdf, .txt**
- Bảng tính: **.xls, .xlsx**

Trình chiếu: **.ppt, .pptx**

Điều này cho thấy sample tập trung vào các file tài liệu và hình ảnh thay vì xử lý toàn bộ file trên hệ thống.

2.2. Đọc nội dung file

Sau khi tìm thấy file phù hợp, sample sử dụng:

None

```
CreateFileW  
GetFileSize  
ReadFile  
CloseHandle
```

để mở và đọc nội dung file vào memory.

Flow:

None

```
Desktop  
|  
▼  
FindFirstFileW / FindNextFileW  
|  
▼  
Check Extension
```



Như vậy sample không chỉ thu thập metadata mà thực sự đọc nội dung của các file mục tiêu.

3. Cơ chế Mã hóa

3.1. Sinh AES key

Sample gọi:

```
None  
CryptAcquireContextW  
CryptGenRandom (Length: 0x10 bytes = 16 bytes = 128 bits)
```

Mô hình:

```
None  
CryptGenRandom → 16-byte Random Key → AES-128 Encryption  
Routine
```

Không có bằng chứng cho thấy AES key được hard-code cố định trong sample.

3.2. AES-128

Sample chứa một implementation AES riêng.

Key schedule có các nhánh tương ứng với:

- 128-bit key —► 10 rounds (Được sử dụng thực tế)
- 192-bit key —► 12 rounds
- 256-bit key —► 14 rounds

Code thực tế sử dụng kích thước:

Do đó thuật toán được sử dụng là:

AES-128

3.3. AES-CBC

Routine encryption xử lý dữ liệu theo block 16 byte.

Trước khi mã hóa mỗi block, sample thực hiện XOR với chaining value:

None

$C1 = \text{AES}(P1 \text{ XOR } IV)$

$C2 = \text{AES}(P2 \text{ XOR } C1)$

$C3 = \text{AES}(P3 \text{ XOR } C2)$

Đây là cơ chế của:

AES-128-CBC

3.4. IV

Sample chứa IV cố định 16 byte:

None

Hex: 03 05 07 09 0b 0d 11 15 1e 45 55 34 49 31 20 42

String: 030507090b0d11151e45553449312042

IV này nằm trực tiếp trong binary tại vùng dữ liệu được sample sử dụng bởi encryption routine.

3.5. Padding

Sample cấp phát buffer được căn theo block size 16 byte:

None

```
padded_size = ((size - 1) / 16 + 1) * 16
```

Sample căn dữ liệu tới biên 16 byte để phù hợp với AES-CBC; không có đủ bằng chứng để gọi phần padding này là PKCS#7.

4. Bảo vệ AES Key bằng Khóa bất đối xứng RSA

Sample chứa public-key parameters trong binary.

Do đó cơ chế tổng thể là:

None

Random AES-128 Key (16 bytes)

|

▼

RSA-512 Public Key Operation (N = 512 bits, e = 65537)

|

▼

Encrypted Key (64 bytes)

|

▼

128 Hexadecimal ASCII Characters

Mô hình Mã hóa Lai (Hybrid Encryption)

Sample sử dụng mô hình **hybrid encryption**:

None

```
[ File Nội Dung ] → AES-128-CBC → Encrypted File Payload  
[ AES Key (16B) ] → RSA-512      → Encrypted AES Key (128 Hex  
Chars)
```

5. Phân tích Lưu lượng Mạng (Network Analysis)

Kết nối Server C2

PCAP cho thấy sample host:

- **Sample Host IP:** 192.168.50.250
- **Destination C2 Server:** 192.168.50.175:9999/TCP

Traffic này được phân biệt với các traffic HTTP/HTTPS khác trong PCAP.

Sample cũng chứa địa chỉ: 192.168.50.175 trong binary.

Do đó có sự tương quan giữa:

None

Static Analysis String: 192.168.50.175

|

▼ (Khớp hoàn toàn)

PCAP Traffic Stream: 192.168.50.250:xxxx →
192.168.50.175:9999

6. Giao thức Truyền Tập Tùy chỉnh (Custom Protocol)

Sample không sử dụng HTTP/HTTPS cho việc truyền file.

Mỗi file bắt đầu bằng header dạng:

None

File:<filename>|Len:<length>!

Header được truyền dưới dạng UTF-16LE.

Ví dụ:

None

File:pcap.pcapng|Len:50932!

Sau header là:

128 ASCII hexadecimal characters

tương ứng 64 byte.

Phần này phù hợp với RSA-512 encrypted AES key.

Sau đó là binary ciphertext của file.

Có thể biểu diễn protocol như sau:

None

+-----+
| Header: File:<filename>|Len:<length>! (UTF-16LE) |
+-----+
| Encrypted AES Key: 128 ASCII Hex Chars (RSA-512) |
+-----+
| Payload: AES-128-CBC Ciphertext (Binary) |
+-----+

7. Danh sách Dữ liệu Thực tế bị Rò rỉ

PCAP cho thấy sample đã gửi ít nhất các file sau:

Tên Tập (File Name)	Kích thước Gốc (Original Length)
pcap.pcapng	50,932 bytes
1MB.txt	65 bytes
exa.xlsx	8,247 bytes
NOTMETOO.docx	11,394 bytes

Tên Tập (File Name)	Kích thước Gốc (Original Length)
sample.pdf	3,028 bytes
TheFlag.png	17,439 bytes

Đây là bằng chứng trực tiếp cho thấy sample thực sự thực hiện file collection và network transfer.

8. Đối chiếu Kích thước Ciphertext

Ví dụ:

None

Tập mẫu: pcap.pcapng

- Kích thước gốc: 50,932 bytes
- Kích thước AES block: 50,944 bytes (Căn lề 16-byte)

Trong TCP stream, cấu trúc transfer tương ứng với:

None

[UTF-16LE Header] + [128 ASCII Hex Chars] + [50,944-byte Ciphertext]

Các file còn lại cũng có kích thước ciphertext phù hợp với việc căn theo AES block size.

Điều này củng cố kết luận rằng phần payload sau RSA-wrapped key chính là AES ciphertext của file.

9. Tổng kết Luồng Xử lý của Mã độc

Từ static analysis và PCAP, có thể xác định:

1. Đầu vào (Input)

Các file trong Desktop có extension:

Các tệp nằm trên Desktop có phần mở rộng mục tiêu: **png, jpg, xls, doc, docx, pdf, ppt, pptx, xlsx, bmp, txt**

2. Xử lý (Processing)

None

Đọc nội dung tệp → Sinh AES-128 Key ngẫu nhiên → Mã hóa AES-128-CBC → Mã hóa AES Key bằng RSA-512

3. Đầu ra (Output)

None

Đóng gói Custom TCP Protocol → Truyền tới Server C2 (192.168.50.175:9999)

10. Chỉ số Nhận diện Độc hại (IOC)

File IOC

None

e91a4af62987547438341666daa943a1ce1a5dd0be0a5ce82111849912bf4e46

Network IOC

None

192.168.50.175:9999/TCP

Protocol IOC

None

File:<filename>|Len:<length>!

Cryptographic IOC

None

AES-128-CBC (IV: 030507090b0d11151e45553449312042)

RSA-512 (Exponent: 65537)

11. Phương pháp Phát hiện (Detection Rules)

11.1. Network detection

Có thể phát hiện kết nối tới:

192.168.50.175:9999

Đồng thời kiểm tra payload có pattern:

File:

và:

Len:<number>!

None

TCP destination port == 9999

AND payload contains "File:"

AND payload matches "Len:[0-9]+!"

Điều này phù hợp để xây dựng IDS/IPS signature hoặc network monitoring rule.

11.2. Endpoint detection

Có thể correlation các hành vi:

None

Process Execution

└─ SHGetSpecialFolderPathW (CSIDL = 0)

└─ FindFirstFileW / FindNextFileW

└─ CreateFileW / ReadFile

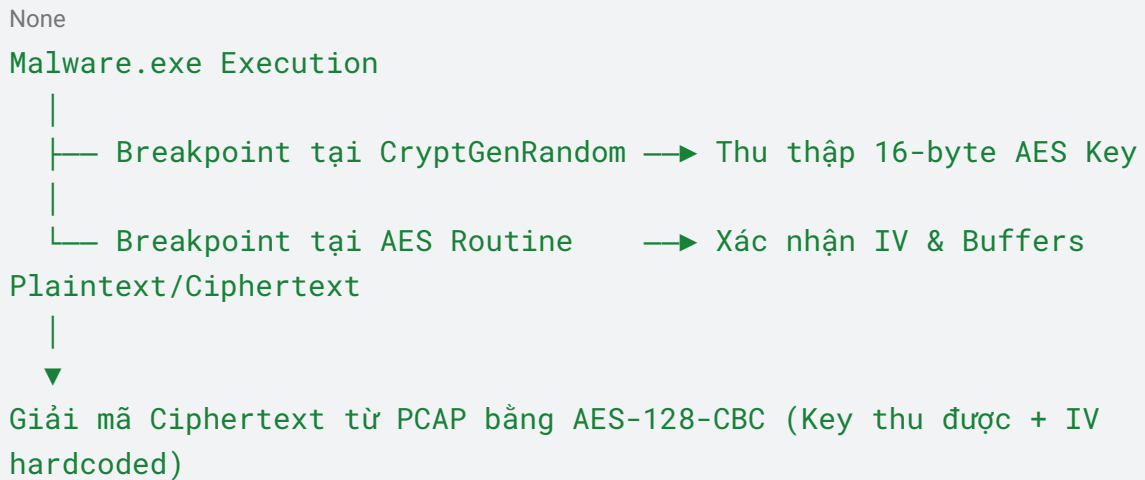
└─ CryptGenRandom

└─ Outbound TCP Connection (Port 9999)

Một process thực hiện đồng thời các hành vi trên có mức độ đáng chú ý cao hơn việc chỉ phát hiện một API riêng lẻ.

12. Phương án Khôi phục & Giải mã (Recovery / Decryption)

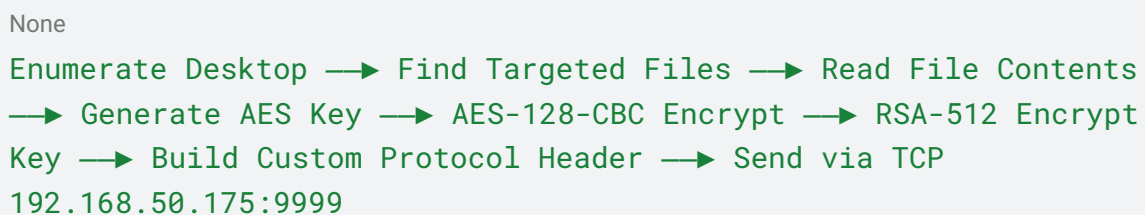
Quy trình:



13. Kết luận

Sample **Malware.exe** thực hiện hành vi thu thập và truyền dữ liệu từ máy nạn nhân.

Chuỗi hành vi được xác định:



Các bằng chứng static và network nhất quán với nhau:

- Binary chứa danh sách extension mục tiêu.
- Binary sử dụng API để enumerate và đọc file.
- Binary chứa implementation AES với 128-bit key.
- Binary sinh AES key bằng **CryptGenRandom**.
- Binary chứa RSA-512 public key với exponent **65537**.

- PCAP xác nhận kết nối tới 192.168.50.175:9999.
- PCAP chứa các header File:<filename>|Len:<length>!.
- PCAP cho thấy nhiều file thực tế được truyền.
- Payload sau header không còn mang định dạng file gốc và có kích thước phù hợp với AES block alignment.

Đánh giá chung (Assessment)

Dựa trên bằng chứng quan sát được, sample có chức năng chính là:

Thu thập các file tài liệu/hình ảnh từ Desktop, mã hóa nội dung bằng AES-128-CBC, bảo vệ AES key bằng RSA-512 và truyền dữ liệu đã mã hóa tới một server thông qua custom TCP protocol tại 192.168.50.175:9999.

Chưa có đủ bằng chứng trong phân tích hiện tại để kết luận sample có persistence, privilege escalation, lateral movement hoặc ransomware behavior. Các kết luận trên chỉ bao gồm những hành vi đã được xác nhận bằng static analysis và PCAP.